Chapter 7. Data Generation, Manipulation, and Conversion

As I mentioned in the preface, this book strives to teach generic SQL techniques that can be applied across multiple database servers. This chapter, however, deals with the generation, conversion, and manipulation of string, numeric, and temporal data, and the SQL language does not include commands covering this functionality. Rather, built-in functions are used to facilitate data generation, conversion, and manipulation, and while the SQL standard does specify some functions, the database vendors often do not comply with the function specifications.

Therefore, my approach for this chapter is to show you some of the common ways in which data is generated and manipulated within SQL statements and then demonstrate some of the built-in functions implemented by Microsoft SQL Server, Oracle Database, and MySQL. Along with reading this chapter, I strongly recommend you download a reference guide covering all the functions implemented by your server. If you work with more than one database server, there are several reference guides that cover multiple servers, such as Kevin Kline et al.'s *SQL in a Nutshell* and Jonathan Gennick's *SQL Pocket Guide*, both from O'Reilly.

Working with String Data

When working with string data, you will be using one of the following character data types:

CHAR

: Holds fixed-length, blank-padded strings. MySQL allows `CHAR` values up to 255 characters in length, Oracle Database permits up to 2,000 characters, and SQL Server allows up to 8,000 characters.

varchar

: Holds variable-length strings. MySQL permits up to 65,535 characters in a `varchar` column, Oracle Database (via the `varchar2` type) allows up to 4,000

characters, and SQL Server allows up to 8,000 characters.

`text` *(MySQL and SQL Server) or* `clob` *(Oracle Database)*

Holds very large variable-length strings (generally referred to as documents in this context). MySQL has multiple text types (`tinytext`, `text`, `mediumtext`, and `longtext`) for documents up to 4 GB in size. SQL Server has a single `text` type for documents up to 2 GB in size, and Oracle Database includes the `clob` data type, which can hold documents up to a whopping 128 TB. SQL Server 2005 also includes the `varchar(max)` data type and recommends its use instead of the `text` type, which will be removed from the server in some future release.

To demonstrate how you can use these various types, I use the following table for some of the examples in this section:

```
CREATE TABLE string_tbl
 (char_fld CHAR(30),
  vchar_fld VARCHAR(30),
  text_fld TEXT
 );
```

The next two subsections show how you can generate and manipulate string data.

String Generation

The simplest way to populate a character column is to enclose a string in quotes, as in the following examples:

```
mysql> INSERT INTO string_tbl (char_fld, vchar_fld, text_fld)
    -> VALUES ('This is char data',
    ->   'This is varchar data',
    ->   'This is text data');
Query OK, 1 row affected (0.00 sec)
```

When inserting string data into a table, remember that if the length of the string exceeds the maximum size for the character column (either the designated maximum or the maximum allowed for the data type), the server will throw an exception. Although this is the default behavior for all three servers, you can configure MySQL and SQL Server to silently truncate the string instead of throwing an exception. To demonstrate how MySQL handles this situation, the following `update` statement attempts to modify

the `vchar_fld` column, whose maximum length is defined as 30, with a string that is 46 characters in length:

```
mysql> UPDATE string_tbl
    -> SET vchar_fld = 'This is a piece of extremely long varchar data';
ERROR 1406 (22001): Data too long for column 'vchar_fld' at row 1
```

Since MySQL 6.0, the default behavior is now "strict" mode, which means that exceptions are thrown when problems arise, whereas in older versions of the server the string would have been truncated and a warning issued. If you would rather have the engine truncate the string and issue a warning instead of raising an exception, you can opt to be in ANSI mode. The following example shows how to check which mode you are in and then how to change the mode using the `set` command:

```
mysql> SELECT @@session.sql_mode;
+-----------------------------------------------------------+
| @@session.sql_mode                                        |
+-----------------------------------------------------------+
| STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION |
+-----------------------------------------------------------+
1 row in set (0.00 sec)

mysql> SET sql_mode='ansi';
Query OK, 0 rows affected (0.08 sec)

mysql> SELECT @@session.sql_mode;
+-------------------------------------------------------------------------------+
| @@session.sql_mode                                                            |
+-------------------------------------------------------------------------------+
| REAL_AS_FLOAT,PIPES_AS_CONCAT,ANSI_QUOTES,IGNORE_SPACE,ONLY_FULL_GROUP_BY,ANSI |
+-------------------------------------------------------------------------------+
1 row in set (0.00 sec)
```

If you rerun the previous `update` statement, you will find that the column has been modified, but the following warning is generated:

```
mysql> SHOW WARNINGS;
+---------+------+-----------------------------------------------+
| Level   | Code | Message                                       |
+---------+------+-----------------------------------------------+
| Warning | 1265 | Data truncated for column 'vchar_fld' at row 1 |
```

```
+---------+------+-------------------------------------------------+
1 row in set (0.00 sec)
```

If you retrieve the `vchar_fld` column, you will see that the string has indeed been truncated:

```
mysql> SELECT vchar_fld
    -> FROM string_tbl;
+------------------------------+
| vchar_fld                    |
+------------------------------+
| This is a piece of extremely l |
+------------------------------+
1 row in set (0.05 sec)
```

As you can see, only the first 30 characters of the 46-character string made it into the `vchar_fld` column. The best way to avoid string truncation (or exceptions, in the case of Oracle Database or MySQL in strict mode) when working with `varchar` columns is to set the upper limit of a column to a high enough value to handle the longest strings that might be stored in the column (keeping in mind that the server allocates only enough space to store the string, so it is not wasteful to set a high upper limit for `varchar` columns).

Including single quotes

Since strings are demarcated by single quotes, you will need to be alert for strings that include single quotes or apostrophes. For example, you won't be able to insert the following string because the server will think that the apostrophe in the word *doesn't* marks the end of the string:

```
UPDATE string_tbl
SET text_fld = 'This string doesn't work';
```

To make the server ignore the apostrophe in the word *doesn't*, you will need to add an *escape* to the string so that the server treats the apostrophe like any other character in the string. All three servers allow you to escape a single quote by adding another single quote directly before, as in:

```
mysql> UPDATE string_tbl
    -> SET text_fld = 'This string didn''t work, but it does now';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

NOTE

Oracle Database and MySQL users may also choose to escape a single quote by adding a backslash character immediately before, as in:

```
UPDATE string_tbl SET text_fld =
 'This string didn\'t work, but it does now'
```

If you retrieve a string for use in a screen or report field, you don't need to do anything special to handle embedded quotes:

```
mysql> SELECT text_fld
    -> FROM string_tbl;
+----------------------------------------+
| text_fld                               |
+----------------------------------------+
| This string didn't work, but it does now |
+----------------------------------------+
1 row in set (0.00 sec)
```

However, if you are retrieving the string to add to a file that another program will read, you may want to include the escape as part of the retrieved string. If you are using MySQL, you can use the built-in function `quote()`, which places quotes around the entire string *and* adds escapes to any single quotes/apostrophes within the string. Here's what our string looks like when retrieved via the `quote()` function:

```
mysql> SELECT quote(text_fld)
    -> FROM string_tbl;
+-----------------------------------------+
| QUOTE(text_fld)                         |
+-----------------------------------------+
| 'This string didn\'t work, but it does now' |
+-----------------------------------------+
1 row in set (0.04 sec)
```

When retrieving data for data export, you may want to use the `quote()` function for all non-system-generated character columns, such as a `customer_notes` column.

Including special characters

If your application is multinational in scope, you might find yourself working with strings that include characters that do not appear on your keyboard. When working with the French and German languages, for example, you might need to include accented characters such as é and ö. The SQL Server and MySQL servers include the built-in function `char()` so that you can build strings from any of the 255 characters in the ASCII character set (Oracle Database users can use the `chr()` function). To demonstrate, the next example retrieves a typed string and its equivalent built via individual characters:

```
mysql> SELECT 'abcdefg', CHAR(97,98,99,100,101,102,103);
+---------+-----------------------------------+
| abcdefg | CHAR(97,98,99,100,101,102,103) |
+---------+-----------------------------------+
| abcdefg | abcdefg                           |
+---------+-----------------------------------+
1 row in set (0.01 sec)
```

Thus, the 97th character in the ASCII character set is the letter *a*. While the characters shown in the preceding example are not special, the following examples show the location of the accented characters along with other special characters, such as currency symbols:

```
mysql> SELECT CHAR(128,129,130,131,132,133,134,135,136,137);
+-------------------------------------------------+
| CHAR(128,129,130,131,132,133,134,135,136,137) |
+-------------------------------------------------+
| Çüéâäàåçêë                                      |
+-------------------------------------------------+
1 row in set (0.01 sec)

mysql> SELECT CHAR(138,139,140,141,142,143,144,145,146,147);
+-------------------------------------------------+
| CHAR(138,139,140,141,142,143,144,145,146,147) |
+-------------------------------------------------+
| èïîìÄÅÉæÆô                                      |
+-------------------------------------------------+
1 row in set (0.01 sec)
```

```
mysql> SELECT CHAR(148,149,150,151,152,153,154,155,156,157);
+-------------------------------------------+
| CHAR(148,149,150,151,152,153,154,155,156,157) |
+-------------------------------------------+
| öòûùÿÖÜø£Ø                                 |
+-------------------------------------------+
1 row in set (0.00 sec)

mysql> SELECT CHAR(158,159,160,161,162,163,164,165);
+-----------------------------------+
| CHAR(158,159,160,161,162,163,164,165) |
+-----------------------------------+
| ×ƒáíóúñÑ                          |
+-----------------------------------+
1 row in set (0.01 sec)
```

NOTE

I am using the `utf8mb4` character set for the examples in this section. If your session is configured for a different character set, you will see a different set of characters than what is shown here. The same concepts apply, but you will need to familiarize yourself with the layout of your character set to locate specific characters.

Building strings character by character can be quite tedious, especially if only a few of the characters in the string are accented. Fortunately, you can use the `concat()` function to concatenate individual strings, some of which you can type while others you can generate via the `char()` function. For example, the following shows how to build the phrase *danke schön* using the `concat()` and `char()` functions:

```
mysql> SELECT CONCAT('danke sch', CHAR(148), 'n');
+-----------------------------------+
| CONCAT('danke sch', CHAR(148), 'n') |
+-----------------------------------+
| danke schön                       |
+-----------------------------------+
1 row in set (0.00 sec)
```

NOTE

Oracle Database users can use the concatenation operator (||) instead of the concat() function, as in:

```
SELECT 'danke sch' || CHR(148) || 'n'
FROM dual;
```

SQL Server does not include a concat() function, so you will need to use the concatenation operator (+), as in:

```
SELECT 'danke sch' + CHAR(148) + 'n'
```

If you have a character and need to find its ASCII equivalent, you can use the ascii() function, which takes the leftmost character in the string and returns a number:

```
mysql> SELECT ASCII('ö');
+------------+
| ASCII('ö') |
+------------+
|        148 |
+------------+
1 row in set (0.00 sec)
```

Using the char() , ascii() , and concat() functions (or concatenation operators), you should be able to work with any Roman language even if you are using a keyboard that does not include accented or special characters.

String Manipulation

Each database server includes many built-in functions for manipulating strings. This section explores two types of string functions: those that return numbers and those that return strings. Before I begin, however, I reset the data in the string_tbl table to the following:

```
mysql> DELETE FROM string_tbl;
Query OK, 1 row affected (0.02 sec)

mysql> INSERT INTO string_tbl (char_fld, vchar_fld, text_fld)
```

```
    -> VALUES ('This string is 28 characters',
    ->   'This string is 28 characters',
    ->   'This string is 28 characters');
Query OK, 1 row affected (0.00 sec)
```

String functions that return numbers

Of the string functions that return numbers, one of the most commonly used is the `length()` function, which returns the number of characters in the string (SQL Server users will need to use the `len()` function). The following query applies the `length()` function to each column in the `string_tbl` table:

```
mysql> SELECT LENGTH(char_fld) char_length,
    ->   LENGTH(vchar_fld) varchar_length,
    ->   LENGTH(text_fld) text_length
    -> FROM string_tbl;
+-------------+----------------+-------------+
| char_length | varchar_length | text_length |
+-------------+----------------+-------------+
|          28 |             28 |          28 |
+-------------+----------------+-------------+
1 row in set (0.00 sec)
```

While the lengths of the `varchar` and `text` columns are as expected, you might have expected the length of the `char` column to be 30, since I told you that strings stored in `char` columns are right-padded with spaces. The MySQL server removes trailing spaces from `char` data when it is retrieved, however, so you will see the same results from all string functions regardless of the type of column in which the strings are stored.

Along with finding the length of a string, you might want to find the location of a substring within a string. For example, if you want to find the position at which the string `'characters'` appears in the `vchar_fld` column, you could use the `position()` function, as demonstrated by the following:

```
mysql> SELECT POSITION('characters' IN vchar_fld)
    -> FROM string_tbl;
+-----------------------------------+
| POSITION('characters' IN vchar_fld) |
+-----------------------------------+
|                                19 |
```

```
+-----------------------------------+
1 row in set (0.12 sec)
```

If the substring cannot be found, the `position()` function returns `0`.

WARNING

For those of you who program in a language such as C or C++, where the first element of an array is at position 0, remember when working with databases that the first character in a string is at position 1. A return value of `0` from `instr()` indicates that the substring could not be found, not that the substring was found at the first position in the string.

If you want to start your search at something other than the first character of your target string, you will need to use the `locate()` function, which is similar to the `position()` function except that it allows an optional third parameter, which is used to define the search's start position. The `locate()` function is also proprietary, whereas the `position()` function is part of the SQL:2003 standard. Here's an example asking for the position of the string `'is'` starting at the fifth character in the `vchar_fld` column:

```
mysql> SELECT LOCATE('is', vchar_fld, 5)
    -> FROM string_tbl;
+---------------------------+
| LOCATE('is', vchar_fld, 5) |
+---------------------------+
|                        13 |
+---------------------------+
1 row in set (0.02 sec)
```

NOTE

Oracle Database does not include the `position()` or `locate()` function, but it does include the `instr()` function, which mimics the `position()` function when provided with two arguments and mimics the `locate()` function when provided with three arguments. SQL Server also doesn't include a `position()` or `locate()` function, but it does include the `charindx()` function, which also accepts either two or three arguments similar to Oracle's `instr()` function.

Another function that takes strings as arguments and returns numbers is the string comparison function `strcmp()`. `strcmp()`, which is implemented only by MySQL and has no

analog in Oracle Database or SQL Server, takes two strings as arguments and returns one of the following:

- `–1` if the first string comes before the second string in sort order
- `0` if the strings are identical
- `1` if the first string comes after the second string in sort order

To illustrate how the function works, I first show the sort order of five strings using a query and then show how the strings compare to one another using `strcmp()`. Here are the five strings that I insert into the `string_tbl` table:

```
mysql> DELETE FROM string_tbl;
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO string_tbl(vchar_fld)
    -> VALUES ('abcd'),
    ->        ('xyz'),
    ->        ('QRSTUV'),
    ->        ('qrstuv'),
    ->        ('12345');
Query OK, 5 rows affected (0.05 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Here are the five strings in their sort order:

```
mysql> SELECT vchar_fld
    -> FROM string_tbl
    -> ORDER BY vchar_fld;
+-----------+
| vchar_fld |
+-----------+
| 12345     |
| abcd      |
| QRSTUV    |
| qrstuv    |
| xyz       |
+-----------+
5 rows in set (0.00 sec)
```

The next query makes six comparisons among the five different strings:

```
mysql> SELECT STRCMP('12345','12345') 12345_12345,
    ->   STRCMP('abcd','xyz') abcd_xyz,
    ->   STRCMP('abcd','QRSTUV') abcd_QRSTUV,
    ->   STRCMP('qrstuv','QRSTUV') qrstuv_QRSTUV,
    ->   STRCMP('12345','xyz') 12345_xyz,
    ->   STRCMP('xyz','qrstuv') xyz_qrstuv;
+-------------+----------+-------------+--------------+-----------+------------+
| 12345_12345 | abcd_xyz | abcd_QRSTUV | qrstuv_QRSTUV | 12345_xyz | xyz_qrstuv |
+-------------+----------+-------------+--------------+-----------+------------+
|           0 |       -1 |          -1 |            0 |        -1 |          1 |
+-------------+----------+-------------+--------------+-----------+------------+
1 row in set (0.00 sec)
```

The first comparison yields `0`, which is to be expected since I compared a string to itself. The fourth comparison also yields `0`, which is a bit surprising, since the strings are composed of the same letters, with one string all uppercase and the other all lowercase. The reason for this result is that MySQL's `strcmp()` function is case-insensitive, which is something to remember when using the function. The other four comparisons yield either `-1` or `1` depending on whether the first string comes before or after the second string in sort order. For example, `strcmp('abcd','xyz')` yields `-1`, since the string `'abcd'` comes before the string `'xyz'`.

Along with the `strcmp()` function, MySQL also allows you to use the `like` and `regexp` operators to compare strings in the `select` clause. Such comparisons will yield `1` (for `true`) or `0` (for `false`). Therefore, these operators allow you to build expressions that return a number, much like the functions described in this section. Here's an example using `like`:

```
mysql> SELECT name, name LIKE '%y' ends_in_y
    -> FROM category;
+-------------+-----------+
| name        | ends_in_y |
+-------------+-----------+
| Action      |         0 |
| Animation   |         0 |
| Children    |         0 |
| Classics    |         0 |
| Comedy      |         1 |
| Documentary |         1 |
| Drama       |         0 |
| Family      |         1 |
| Foreign     |         0 |
```

```
| Games       |         0 |
| Horror      |         0 |
| Music       |         0 |
| New         |         0 |
| Sci-Fi      |         0 |
| Sports      |         0 |
| Travel      |         0 |
+-------------+-----------+
16 rows in set (0.00 sec)
```

This example retrieves all the category names, along with an expression that returns `1` if the name ends in "y" or `0` otherwise. If you want to perform more complex pattern matches, you can use the `regexp` operator, as demonstrated by the following:

```
mysql> SELECT name, name REGEXP 'y$' ends_in_y
    -> FROM category;
+-------------+-----------+
| name        | ends_in_y |
+-------------+-----------+
| Action      |         0 |
| Animation   |         0 |
| Children    |         0 |
| Classics    |         0 |
| Comedy      |         1 |
| Documentary |         1 |
| Drama       |         0 |
| Family      |         1 |
| Foreign     |         0 |
| Games       |         0 |
| Horror      |         0 |
| Music       |         0 |
| New         |         0 |
| Sci-Fi      |         0 |
| Sports      |         0 |
| Travel      |         0 |
+-------------+-----------+
16 rows in set (0.00 sec)
```

The second column of this query returns `1` if the value stored in the `name` column matches the given regular expression.

NOTE

Microsoft SQL Server and Oracle Database users can achieve similar results by building `case` expressions, which I describe in detail in Chapter 11.

String functions that return strings

In some cases, you will need to modify existing strings, either by extracting part of the string or by adding additional text to the string. Every database server includes multiple functions to help with these tasks. Before I begin, I once again reset the data in the `string_tbl` table:

```
mysql> DELETE FROM string_tbl;
Query OK, 5 rows affected (0.00 sec)

mysql> INSERT INTO string_tbl (text_fld)
    -> VALUES ('This string was 29 characters');
Query OK, 1 row affected (0.01 sec)
```

Earlier in the chapter, I demonstrated the use of the `concat()` function to help build words that include accented characters. The `concat()` function is useful in many other situations, including when you need to append additional characters to a stored string. For instance, the following example modifies the string stored in the `text_fld` column by tacking an additional phrase on the end:

```
mysql> UPDATE string_tbl
    -> SET text_fld = CONCAT(text_fld, ', but now it is longer');
Query OK, 1 row affected (0.03 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

The contents of the `text_fld` column are now as follows:

```
mysql> SELECT text_fld
    -> FROM string_tbl;
+-------------------------------------------------+
| text_fld                                        |
+-------------------------------------------------+
| This string was 29 characters, but now it is longer |
+-------------------------------------------------+
1 row in set (0.00 sec)
```

Thus, like all functions that return a string, you can use `concat()` to replace the data stored in a character column.

Another common use for the `concat()` function is to build a string from individual pieces of data. For example, the following query generates a narrative string for each customer:

```
mysql> SELECT concat(first_name, ' ', last_name,
    ->   ' has been a customer since ', date(create_date)) cust_narrative
    -> FROM customer;
+-----------------------------------------------------+
| cust_narrative                                      |
+-----------------------------------------------------+
| MARY SMITH has been a customer since 2006-02-14     |
| PATRICIA JOHNSON has been a customer since 2006-02-14 |
| LINDA WILLIAMS has been a customer since 2006-02-14 |
| BARBARA JONES has been a customer since 2006-02-14  |
| ELIZABETH BROWN has been a customer since 2006-02-14 |
| JENNIFER DAVIS has been a customer since 2006-02-14 |
| MARIA MILLER has been a customer since 2006-02-14   |
| SUSAN WILSON has been a customer since 2006-02-14   |
| MARGARET MOORE has been a customer since 2006-02-14 |
| DOROTHY TAYLOR has been a customer since 2006-02-14 |
...
| RENE MCALISTER has been a customer since 2006-02-14 |
| EDUARDO HIATT has been a customer since 2006-02-14  |
| TERRENCE GUNDERSON has been a customer since 2006-02-14 |
| ENRIQUE FORSYTHE has been a customer since 2006-02-14 |
| FREDDIE DUGGAN has been a customer since 2006-02-14 |
| WADE DELVALLE has been a customer since 2006-02-14  |
| AUSTIN CINTRON has been a customer since 2006-02-14 |
+-----------------------------------------------------+
599 rows in set (0.00 sec)
```

The `concat()` function can handle any expression that returns a string and will even convert numbers and dates to string format, as evidenced by the date column (`create_date`) used as an argument. Although Oracle Database includes the `concat()` function, it will accept only two string arguments, so the previous query will not work on Oracle. Instead, you would need to use the concatenation operator (`||`) rather than a function call, as in:

```
SELECT first_name || ' ' || last_name ||
  ' has been a customer since ' || date(create_date)) cust_narrative
```

```
FROM customer;
```

SQL Server does not include a `concat()` function, so you would need to use the same approach as the previous query, except that you would use SQL Server's concatenation operator (`+`) instead of `||` .

While `concat()` is useful for adding characters to the beginning or end of a string, you may also have a need to add or replace characters in the *middle* of a string. All three database servers provide functions for this purpose, but all of them are different, so I demonstrate the MySQL function and then show the functions from the other two servers.

MySQL includes the `insert()` function, which takes four arguments: the original string, the position at which to start, the number of characters to replace, and the replacement string. Depending on the value of the third argument, the function may be used to either insert or replace characters in a string. With a value of `0` for the third argument, the replacement string is inserted, and any trailing characters are pushed to the right, as in:

```
mysql> SELECT INSERT('goodbye world', 9, 0, 'cruel ') string;
+---------------------+
| string              |
+---------------------+
| goodbye cruel world |
+---------------------+
1 row in set (0.00 sec)
```

In this example, all characters starting from position 9 are pushed to the right, and the string `'cruel'` is inserted. If the third argument is greater than zero, then that number of characters is replaced with the replacement string, as in:

```
mysql> SELECT INSERT('goodbye world', 1, 7, 'hello') string;
+-------------+
| string      |
+-------------+
| hello world |
+-------------+
1 row in set (0.00 sec)
```

For this example, the first seven characters are replaced with the string `'hello'` . Oracle Database does not provide a single function with the flexibility of MySQL's `insert()`

function, but Oracle does provide the `replace()` function, which is useful for replacing one substring with another. Here's the previous example reworked to use `replace()`:

```
SELECT REPLACE('goodbye world', 'goodbye', 'hello')
FROM dual;
```

All instances of the string `'goodbye'` will be replaced with the string `'hello'`, resulting in the string `'hello world'`. The `replace()` function will replace *every* instance of the search string with the replacement string, so you need to be careful that you don't end up with more replacements than you anticipated.

SQL Server also includes a `replace()` function with the same functionality as Oracle's, but SQL Server also includes a function called `stuff()` with similar functionality to MySQL's `insert()` function. Here's an example:

```
SELECT STUFF('hello world', 1, 5, 'goodbye cruel')
```

When executed, five characters are removed starting at position 1, and then the string `'goodbye cruel'` is inserted at the starting position, resulting in the string `'goodbye cruel world'`.

Along with inserting characters into a string, you may have a need to *extract* a substring from a string. For this purpose, all three servers include the `substring()` function (although Oracle Database's version is called `substr()`), which extracts a specified number of characters starting at a specified position. The following example extracts five characters from a string starting at the ninth position:

```
mysql> SELECT SUBSTRING('goodbye cruel world', 9, 5);
+---------------------------------------+
| SUBSTRING('goodbye cruel world', 9, 5) |
+---------------------------------------+
| cruel                                 |
+---------------------------------------+
1 row in set (0.00 sec)
```

Along with the functions demonstrated here, all three servers include many more built-in functions for manipulating string data. While many of them are designed for very specific purposes, such as generating the string equivalent of octal or hexadecimal numbers, there are many other general-purpose functions as well, such as functions that remove

or add trailing spaces. For more information, consult your server's SQL reference guide, or a general-purpose SQL reference guide such as *SQL in a Nutshell* (O'Reilly).

Working with Numeric Data

Unlike string data (and temporal data, as you will see shortly), numeric data generation is quite straightforward. You can type a number, retrieve it from another column, or generate it via a calculation. All the usual arithmetic operators (`+` , `-` , `*` , `/`) are available for performing calculations, and parentheses may be used to dictate precedence, as in:

```
mysql> SELECT (37 * 59) / (78 - (8 * 6));
+---------------------------+
| (37 * 59) / (78 - (8 * 6)) |
+---------------------------+
|                     72.77 |
+---------------------------+
1 row in set (0.00 sec)
```

As I mentioned in Chapter 2, the main concern when storing numeric data is that numbers might be rounded if they are larger than the specified size for a numeric column. For example, the number 9.96 will be rounded to 10.0 if stored in a column defined as `float(3,1)` .

Performing Arithmetic Functions

Most of the built-in numeric functions are used for specific arithmetic purposes, such as determining the square root of a number. Table 7-1 lists some of the common numeric functions that take a single numeric argument and return a number.

Table 7-1. Single-argument numeric functions

Function name	Description
`acos( x )`	Calculates the arc cosine of `x`
`asin( x )`	Calculates the arc sine of `x`
`atan( x )`	Calculates the arc tangent of `x`
`cos( x )`	Calculates the cosine of `x`
`cot( x )`	Calculates the cotangent of `x`
`exp( x )`	Calculates e^x
`ln( x )`	Calculates the natural log of `x`
`sin( x )`	Calculates the sine of `x`
`sqrt( x )`	Calculates the square root of `x`
`tan( x )`	Calculates the tangent of `x`

These functions perform very specific tasks, and I refrain from showing examples for these functions (if you don't recognize a function by name or description, then you probably don't need it). Other numeric functions used for calculations, however, are a bit more flexible and deserve some explanation.

For example, the `modulo` operator, which calculates the remainder when one number is divided into another number, is implemented in MySQL and Oracle Database via the `mod()` function. The following example calculates the remainder when `10` is divided by `4`:

```
mysql> SELECT MOD(10,4);
+-----------+
| MOD(10,4) |
+-----------+
|         2 |
```

```
+-----------+
1 row in set (0.02 sec)
```

While the `mod()` function is typically used with integer arguments, with MySQL you can also use real numbers, as in:

```
mysql> SELECT MOD(22.75, 5);
+---------------+
| MOD(22.75, 5) |
+---------------+
|          2.75 |
+---------------+
1 row in set (0.02 sec)
```

NOTE

SQL Server does not have a `mod()` function. Instead, the operator `%` is used for finding remainders. The expression `10 % 4` will therefore yield the value `2`.

Another numeric function that takes two numeric arguments is the `pow()` function (or `power()` if you are using Oracle Database or SQL Server), which returns one number raised to the power of a second number, as in:

```
mysql> SELECT POW(2,8);
+----------+
| POW(2,8) |
+----------+
|      256 |
+----------+
1 row in set (0.03 sec)
```

Thus, `pow(2,8)` is the MySQL equivalent of specifying 2^8. Since computer memory is allocated in chunks of 2^x bytes, the `pow()` function can be a handy way to determine the exact number of bytes in a certain amount of memory:

```
mysql> SELECT POW(2,10) kilobyte, POW(2,20) megabyte,
    ->   POW(2,30) gigabyte, POW(2,40) terabyte;
+----------+----------+------------+---------------+
| kilobyte | megabyte | gigabyte   | terabyte      |
```

```
+----------+----------+------------+---------------+
|    1024 |  1048576 | 1073741824 | 1099511627776 |
+----------+----------+------------+---------------+
1 row in set (0.00 sec)
```

I don't know about you, but I find it easier to remember that a gigabyte is 2^{30} bytes than to remember the number 1,073,741,824.

Controlling Number Precision

When working with floating-point numbers, you may not always want to interact with or display a number with its full precision. For example, you may store monetary transaction data with a precision to six decimal places, but you might want to round to the nearest hundredth for display purposes. Four functions are useful when limiting the precision of floating-point numbers: `ceil()`, `floor()`, `round()`, and `truncate()`. All three servers include these functions, although Oracle Database includes `trunc()` instead of `truncate()`, and SQL Server includes `ceiling()` instead of `ceil()`.

The `ceil()` and `floor()` functions are used to round either up or down to the closest integer, as demonstrated by the following:

```
mysql> SELECT CEIL(72.445), FLOOR(72.445);
+--------------+---------------+
| CEIL(72.445) | FLOOR(72.445) |
+--------------+---------------+
|           73 |            72 |
+--------------+---------------+
1 row in set (0.06 sec)
```

Thus, any number between 72 and 73 will be evaluated as 73 by the `ceil()` function and 72 by the `floor()` function. Remember that `ceil()` will round up even if the decimal portion of a number is very small, and `floor()` will round down even if the decimal portion is quite significant, as in:

```
mysql> SELECT CEIL(72.000000001), FLOOR(72.999999999);
+--------------------+---------------------+
| CEIL(72.000000001) | FLOOR(72.999999999) |
+--------------------+---------------------+
|                 73 |                  72 |
```

```
+--------------------+---------------------+
1 row in set (0.00 sec)
```

If this is a bit too severe for your application, you can use the `round()` function to round up or down from the *midpoint* between two integers, as in:

```
mysql> SELECT ROUND(72.49999), ROUND(72.5), ROUND(72.50001);
+-----------------+-------------+-----------------+
| ROUND(72.49999) | ROUND(72.5) | ROUND(72.50001) |
+-----------------+-------------+-----------------+
|              72 |          73 |              73 |
+-----------------+-------------+-----------------+
1 row in set (0.00 sec)
```

Using `round()`, any number whose decimal portion is halfway or more between two integers will be rounded up, whereas the number will be rounded down if the decimal portion is anything less than halfway between the two integers.

Most of the time, you will want to keep at least some part of the decimal portion of a number rather than rounding to the nearest integer; the `round()` function allows an optional second argument to specify how many digits to the right of the decimal place to round to. The next example shows how you can use the second argument to round the number 72.0909 to one, two, and three decimal places:

```
mysql> SELECT ROUND(72.0909, 1), ROUND(72.0909, 2), ROUND(72.0909, 3);
+-------------------+-------------------+-------------------+
| ROUND(72.0909, 1) | ROUND(72.0909, 2) | ROUND(72.0909, 3) |
+-------------------+-------------------+-------------------+
|              72.1 |             72.09 |            72.091 |
+-------------------+-------------------+-------------------+
1 row in set (0.00 sec)
```

Like the `round()` function, the `truncate()` function allows an optional second argument to specify the number of digits to the right of the decimal, but `truncate()` simply discards the unwanted digits without rounding. The next example shows how the number 72.0909 would be truncated to one, two, and three decimal places:

```
mysql> SELECT TRUNCATE(72.0909, 1), TRUNCATE(72.0909, 2),
    ->   TRUNCATE(72.0909, 3);
+----------------------+----------------------+----------------------+
```

```
| TRUNCATE(72.0909, 1) | TRUNCATE(72.0909, 2) | TRUNCATE(72.0909, 3) |
+----------------------+----------------------+----------------------+
|                 72.0 |                72.09 |               72.090 |
+----------------------+----------------------+----------------------+
1 row in set (0.00 sec)
```

NOTE

SQL Server does not include a `truncate()` function. Instead, the `round()` function allows for an optional third argument that, if present and nonzero, calls for the number to be truncated rather than rounded.

Both `truncate()` and `round()` also allow a *negative* value for the second argument, meaning that numbers to the *left* of the decimal place are truncated or rounded. This might seem like a strange thing to do at first, but there are valid applications. For example, you might sell a product that can be purchased only in units of 10. If a customer were to order 17 units, you could choose from one of the following methods to modify the customer's order quantity:

```
mysql> SELECT ROUND(17, -1), TRUNCATE(17, -1);
+---------------+-----------------+
| ROUND(17, -1) | TRUNCATE(17, -1) |
+---------------+-----------------+
|            20 |              10 |
+---------------+-----------------+
1 row in set (0.00 sec)
```

If the product in question is thumbtacks, then it might not make much difference to your bottom line whether you sold the customer 10 or 20 thumbtacks when only 17 were requested; if you are selling Rolex watches, however, your business may fare better by rounding.

Handling Signed Data

If you are working with numeric columns that allow negative values (in Chapter 2, I showed how a numeric column may be labeled *unsigned*, meaning that only positive numbers are allowed), several numeric functions might be of use. Let's say, for example, that you are asked to generate a report showing the current status of a set of bank accounts using the following data from the `account` table:

```
+------------+--------------+---------+
| account_id | acct_type    | balance |
+------------+--------------+---------+
|        123 | MONEY MARKET |  785.22 |
|        456 | SAVINGS      |    0.00 |
|        789 | CHECKING     | -324.22 |
+------------+--------------+---------+
```

The following query returns three columns useful for generating the report:

```
mysql> SELECT account_id, SIGN(balance), ABS(balance)
    -> FROM account;
+------------+---------------+--------------+
| account_id | SIGN(balance) | ABS(balance) |
+------------+---------------+--------------+
|        123 |             1 |       785.22 |
|        456 |             0 |         0.00 |
|        789 |            -1 |       324.22 |
+------------+---------------+--------------+
3 rows in set (0.00 sec)
```

The second column uses the `sign()` function to return `-1` if the account balance is negative, `0` if the account balance is zero, and `1` if the account balance is positive. The third column returns the absolute value of the account balance via the `abs()` function.

Working with Temporal Data

Of the three types of data discussed in this chapter (character, numeric, and temporal), temporal data is the most involved when it comes to data generation and manipulation. Some of the complexity of temporal data is caused by the myriad ways in which a single date and time can be described. For example, the date on which I wrote this paragraph can be described in all the following ways:

- Wednesday, June 5, 2019
- 6/05/2019 2:14:56 P.M. EST
- 6/05/2019 19:14:56 GMT
- 1562019 (Julian format)
- Star date [−4] 97026.79 14:14:56 (*Star Trek* format)

While some of these differences are purely a matter of formatting, most of the complexity has to do with your frame of reference, which we explore in the next section.

Dealing with Time Zones

Because people around the world prefer that noon coincides roughly with the sun's peak at their location, there has never been a serious attempt to coerce everyone to use a universal clock. Instead, the world has been sliced into 24 imaginary sections, called *time zones*; within a particular time zone, everyone agrees on the current time, whereas people in different time zones do not. While this seems simple enough, some geographic regions shift their time by one hour twice a year (implementing what is known as *daylight saving time*) and some do not, so the time difference between two points on Earth might be four hours for one-half of the year and five hours for the other half of the year. Even within a single time zone, different regions may or may not adhere to daylight saving time, causing different clocks in the same time zone to agree for one-half of the year but be one hour different for the rest of the year.

While the computer age has exacerbated the issue, people have been dealing with time zone differences since the early days of naval exploration. To ensure a common point of reference for timekeeping, fifteenth-century navigators set their clocks to the time of day in Greenwich, England. This became known as *Greenwich Mean Time*, or GMT. All other time zones can be described by the number of hours' difference from GMT; for example, the time zone for the Eastern United States, known as *Eastern Standard Time*, can be described as GMT –5:00, or five hours earlier than GMT.

Today, we use a variation of GMT called *Coordinated Universal Time*, or UTC, which is based on an atomic clock (or, to be more precise, the average time of 200 atomic clocks in 50 locations worldwide, which is referred to as *Universal Time*). Both SQL Server and MySQL provide functions that will return the current UTC timestamp (`getutcdate()` for SQL Server and `utc_timestamp()` for MySQL).

Most database servers default to the time zone setting of the server on which it resides and provide tools for modifying the time zone if needed. For example, a database used to store stock exchange transactions from around the world would generally be configured to use UTC time, whereas a database used to store transactions at a particular retail establishment might use the server's time zone.

MySQL keeps two different time zone settings: a global time zone and a session time zone, which may be different for each user logged in to a database. You can see both settings via the following query:

```
mysql> SELECT @@global.time_zone, @@session.time_zone;
+-------------------+--------------------+
| @@global.time_zone | @@session.time_zone |
+-------------------+--------------------+
| SYSTEM            | SYSTEM             |
+-------------------+--------------------+
1 row in set (0.00 sec)
```

A value of `system` tells you that the server is using the time zone setting from the server on which the database resides.

If you are sitting at a computer in Zurich, Switzerland, and you open a session across the network to a MySQL server situated in New York, you may want to change the time zone setting for your session, which you can do via the following command:

```
mysql> SET time_zone = 'Europe/Zurich';
Query OK, 0 rows affected (0.18 sec)
```

If you check the time zone settings again, you will see the following:

```
mysql> SELECT @@global.time_zone, @@session.time_zone;
+-------------------+--------------------+
| @@global.time_zone | @@session.time_zone |
+-------------------+--------------------+
| SYSTEM            | Europe/Zurich      |
+-------------------+--------------------+
1 row in set (0.00 sec)
```

All dates displayed in your session will now conform to Zurich time.

NOTE

Oracle Database users can change the time zone setting for a session via the following command:

```
ALTER SESSION TIMEZONE = 'Europe/Zurich'
```

Generating Temporal Data

You can generate temporal data via any of the following means:

- Copying data from an existing `date`, `datetime`, or `time` column
- Executing a built-in function that returns a `date`, `datetime`, or `time`
- Building a string representation of the temporal data to be evaluated by the server

To use the last method, you will need to understand the various components used in formatting dates.

String representations of temporal data

Table 2-4 in Chapter 2 presented the more popular date components; to refresh your memory, Table 7-2 shows these same components.

Table 7-2. Date format components

Component	Definition	Range
YYYY	Year, including century	`1000` to `9999`
MM	Month	01 (January) to 12 (December)
DD	Day	`01` to `31`
HH	Hour	00 to 23
HHH	Hours (elapsed)	`-838` to `838`
MI	Minute	00 to 59
SS	Second	`00` to `59`

To build a string that the server can interpret as a `date`, `datetime`, or `time`, you need to put the various components together in the order shown in Table 7-3.

Table 7-3. Required date components

Type	Default format
`date`	YYYY-MM-DD
`datetime`	YYYY-MM-DD HH:MI:SS
`timestamp`	YYYY-MM-DD HH:MI:SS
`time`	HHH:MI:SS

Thus, to populate a `datetime` column with 3:30 P.M. on September 17, 2019, you will need to build the following string:

```
'2019-09-17 15:30:00'
```

If the server is expecting a `datetime` value, such as when updating a `datetime` column or when calling a built-in function that takes a `datetime` argument, you can provide a properly formatted string with the required date components, and the server will do the conversion for you. For example, here's a statement used to modify the return date of a film rental:

```
UPDATE rental
SET return_date = '2019-09-17 15:30:00'
WHERE rental_id = 99999;
```

The server determines that the string provided in the `set` clause must be a `datetime` value, since the string is being used to populate a `datetime` column. Therefore, the server will attempt to convert the string for you by parsing the string into the six components (year, month, day, hour, minute, second) included in the default `datetime` format.

String-to-date conversions

If the server is *not* expecting a `datetime` value or if you would like to represent the `datetime` using a nondefault format, you will need to tell the server to convert the string to a `datetime`. For example, here is a simple query that returns a `datetime` value using the `cast()` function:

```
mysql> SELECT CAST('2019-09-17 15:30:00' AS DATETIME);
+---------------------------------------+
| CAST('2019-09-17 15:30:00' AS DATETIME) |
+---------------------------------------+
| 2019-09-17 15:30:00                   |
+---------------------------------------+
1 row in set (0.00 sec)
```

We cover the `cast()` function at the end of this chapter. While this example demonstrates how to build `datetime` values, the same logic applies to the `date` and `time` types as well. The following query uses the `cast()` function to generate a `date` value and a `time` value:

```
mysql> SELECT CAST('2019-09-17' AS DATE) date_field,
    ->   CAST('108:17:57' AS TIME) time_field;
+------------+------------+
| date_field | time_field |
+------------+------------+
| 2019-09-17 | 108:17:57  |
+------------+------------+
1 row in set (0.00 sec)
```

You might, of course, explicitly convert your strings even when the server is expecting a `date`, `datetime`, or `time` value, rather than allowing the server to do an implicit conversion.

When strings are converted to temporal values—whether explicitly or implicitly—you must provide all the date components in the required order. While some servers are quite strict regarding the date format, the MySQL server is quite lenient about the separators used between the components. For example, MySQL will accept all of the following strings as valid representations of 3:30 P.M. on September 17, 2019:

```
'2019-09-17 15:30:00'
'2019/09/17 15:30:00'
'2019,09,17,15,30,00'
'20190917153000'
```

Although this gives you a bit more flexibility, you may find yourself trying to generate a temporal value *without* the default date components; the next section demonstrates a built-in function that is far more flexible than the `cast()` function.

Functions for generating dates

If you need to generate temporal data from a string and the string is not in the proper form to use the `cast()` function, you can use a built-in function that allows you to provide a format string along with the date string. MySQL includes the `str_to_date()` function for this purpose. Say, for example, that you pull the string `'September 17, 2019'` from a file and need to use it to update a `date` column. Since the string is not in the required `YYYY-MM-DD` format, you can use `str_to_date()` instead of reformatting the string so that you can use the `cast()` function, as in:

```
UPDATE rental
SET return_date = STR_TO_DATE('September 17, 2019', '%M %d, %Y')
WHERE rental_id = 99999;
```

The second argument in the call to `str_to_date()` defines the format of the date string, with, in this case, a month name (`%M`), a numeric day (`%d`), and a four-digit numeric year (`%Y`). While there are more than 30 recognized format components, Table 7-4 defines the dozen or so of the most commonly used components.

Table 7-4. Date format components

Format component	Description
%M	Month name (January to December)
%m	Month numeric (01 to 12)
%d	Day numeric (01 to 31)
%j	Day of year (001 to 366)
%W	Weekday name (Sunday to Saturday)
%Y	Year, four-digit numeric
%y	Year, two-digit numeric
%H	Hour (00 to 23)
%h	Hour (01 to 12)
%i	Minutes (00 to 59)
%s	Seconds (00 to 59)
%f	Microseconds (000000 to 999999)
%p	A.M. or P.M.

The str_to_date() function returns a datetime , date , or time value depending on the contents of the format string. For example, if the format string includes only %H , %i , and %s , then a time value will be returned.

NOTE

Oracle Database users can use the to_date() function in the same manner as MySQL's str_to_date() function. SQL Server includes a convert() function that is not quite as flexible as MySQL and Oracle Database; rather than supplying a custom format string, your date string must conform to one of 21 predefined formats.

If you are trying to generate the *current* date/time, then you won't need to build a string, because the following built-in functions will access the system clock and return the current date and/or time as a string for you:

```
mysql> SELECT CURRENT_DATE(), CURRENT_TIME(), CURRENT_TIMESTAMP();
+---------------+---------------+---------------------+
| CURRENT_DATE() | CURRENT_TIME() | CURRENT_TIMESTAMP() |
+---------------+---------------+---------------------+
| 2019-06-05    | 16:54:36      | 2019-06-05 16:54:36 |
+---------------+---------------+---------------------+
1 row in set (0.12 sec)
```

The values returned by these functions are in the default format for the temporal type being returned. Oracle Database will include `current_date()` and `current_timestamp()` but not `current_time()`, and Microsoft SQL Server includes only the `current_timestamp()` function.

Manipulating Temporal Data

This section explores the built-in functions that take date arguments and return dates, strings, or numbers.

Temporal functions that return dates

Many of the built-in temporal functions take one date as an argument and return another date. MySQL's `date_add()` function, for example, allows you to add any kind of interval (e.g., days, months, years) to a specified date to generate another date. Here's an example that demonstrates how to add five days to the current date:

```
mysql> SELECT DATE_ADD(CURRENT_DATE(), INTERVAL 5 DAY);
+----------------------------------------+
| DATE_ADD(CURRENT_DATE(), INTERVAL 5 DAY) |
+----------------------------------------+
| 2019-06-10                             |
+----------------------------------------+
1 row in set (0.06 sec)
```

The second argument is composed of three elements: the `interval` keyword, the desired quantity, and the type of interval. Table 7-5 shows some of the commonly used interval types.

Table 7-5. Common interval types

Interval name	Description
second	Number of seconds
minute	Number of minutes
hour	Number of hours
day	Number of days
month	Number of months
year	Number of years
minute_second	Number of minutes and seconds, separated by ":"
hour_second	Number of hours, minutes, and seconds, separated by ":"
year_month	Number of years and months, separated by "-"

While the first six types listed in Table 7-5 are pretty straightforward, the last three types require a bit more explanation since they have multiple elements. For example, if you are told that a film was actually returned 3 hours, 27 minutes, and 11 seconds later than what was originally specified, you can fix it via the following:

```
UPDATE rental
SET return_date = DATE_ADD(return_date, INTERVAL '3:27:11' HOUR_SECOND)
WHERE rental_id = 99999;
```

In this example, the `date_add()` function takes the value in the `return_date` column and adds 3 hours, 27 minutes, and 11 seconds to it. Then it uses the value that results to modify the `return_date` column.

Or, if you work in HR and found out that employee ID 4789 claimed to be older than he actually is, you could add 9 years and 11 months to his birth date, as in:

```
UPDATE employee
SET birth_date = DATE_ADD(birth_date, INTERVAL '9-11' YEAR_MONTH)
```

```
WHERE emp_id = 4789;
```

NOTE

SQL Server users can accomplish the previous example using the `dateadd()` function:

```
UPDATE employee
SET birth_date =
  DATEADD(MONTH, 119, birth_date)
WHERE emp_id = 4789
```

SQL Server doesn't have combined intervals (i.e., `year_month`), so I converted 9 years and 11 months to 119 months.

Oracle Database users can use the `add_months()` function for this example, as in:

```
UPDATE employee
SET birth_date = ADD_MONTHS(birth_date, 119)
WHERE emp_id = 4789;
```

There are some cases where you want to add an interval to a date, and you know where you want to arrive but not how many days it takes to get there. For example, let's say that a bank customer logs on to the online banking system and schedules a transfer for the end of the month. Rather than writing some code that figures out the current month and then looks up the number of days in that month, you can call the `last_day()` function, which does the work for you (both MySQL and Oracle Database include the `last_day()` function; SQL Server has no comparable function). If the customer asks for the transfer on September 17, 2019, you could find the last day of September via the following:

```
mysql> SELECT LAST_DAY('2019-09-17');
+-----------------------+
| LAST_DAY('2019-09-17') |
+-----------------------+
| 2019-09-30            |
+-----------------------+
1 row in set (0.10 sec)
```

Whether you provide a `date` or `datetime` value, the `last_day()` function always returns a `date`. Although this function may not seem like an enormous time-saver, the underlying logic can be tricky if you're trying to find the last day of February and need to figure out whether the current year is a leap year.

Temporal functions that return strings

Most of the temporal functions that return string values are used to extract a portion of a date or time. For example, MySQL includes the `dayname()` function to determine which day of the week a certain date falls on, as in:

```
mysql> SELECT DAYNAME('2019-09-18');
+----------------------+
| DAYNAME('2019-09-18') |
+----------------------+
| Wednesday            |
+----------------------+
1 row in set (0.00 sec)
```

Many such functions are included with MySQL for extracting information from date values, but I recommend that you use the `extract()` function instead, since it's easier to remember a few variations of one function than to remember a dozen different functions. Additionally, the `extract()` function is part of the SQL:2003 standard and has been implemented by Oracle Database as well as MySQL.

The `extract()` function uses the same interval types as the `date_add()` function (see Table 7-5) to define which element of the date interests you. For example, if you want to extract just the year portion of a `datetime` value, you can do the following:

```
mysql> SELECT EXTRACT(YEAR FROM '2019-09-18 22:19:05');
+-----------------------------------------+
| EXTRACT(YEAR FROM '2019-09-18 22:19:05') |
+-----------------------------------------+
|                                    2019 |
+-----------------------------------------+
1 row in set (0.00 sec)
```

NOTE

SQL Server doesn't include an implementation of `extract()`, but it does include the `datepart()` function. Here's how you would extract the year from a `datetime` value using `datepart()`:

```
SELECT DATEPART(YEAR, GETDATE())
```

Temporal functions that return numbers

Earlier in this chapter, I showed you a function used to add a given interval to a date value, thus generating another date value. Another common activity when working with dates is to take *two* date values and determine the number of intervals (days, weeks, years) *between* the two dates. For this purpose, MySQL includes the function `datediff()`, which returns the number of full days between two dates. For example, if I want to know the number of days that my kids will be out of school this summer, I can do the following:

```
mysql> SELECT DATEDIFF('2019-09-03', '2019-06-21');
+-------------------------------------+
| DATEDIFF('2019-09-03', '2019-06-21') |
+-------------------------------------+
|                                  74 |
+-------------------------------------+
1 row in set (0.00 sec)
```

Thus, I will have to endure 74 days of poison ivy, mosquito bites, and scraped knees before the kids are safely back at school. The `datediff()` function ignores the time of day in its arguments. Even if I include a time of day, setting it to one second until midnight for the first date and to one second after midnight for the second date, those times will have no effect on the calculation:

```
mysql> SELECT DATEDIFF('2019-09-03 23:59:59', '2019-06-21 00:00:01');
+-----------------------------------------------------+
| DATEDIFF('2019-09-03 23:59:59', '2019-06-21 00:00:01') |
+-----------------------------------------------------+
|                                                  74 |
+-----------------------------------------------------+
1 row in set (0.00 sec)
```

If I switch the arguments and have the earlier date first, `datediff()` will return a negative number, as in:

```
mysql> SELECT DATEDIFF('2019-06-21', '2019-09-03');
+-------------------------------------+
| DATEDIFF('2019-06-21', '2019-09-03') |
+-------------------------------------+
|                                 -74 |
+-------------------------------------+
1 row in set (0.00 sec)
```

NOTE

SQL Server also includes the `datediff()` function, but it is more flexible than the MySQL implementation in that you can specify the interval type (i.e., year, month, day, hour) instead of counting only the number of days between two dates. Here's how SQL Server would accomplish the previous example:

```
SELECT DATEDIFF(DAY, '2019-06-21', '2019-09-03')
```

Oracle Database allows you to determine the number of days between two dates simply by subtracting one date from another.

Conversion Functions

Earlier in this chapter, I showed you how to use the `cast()` function to convert a string to a `datetime` value. While every database server includes a number of proprietary functions used to convert data from one type to another, I recommend using the `cast()` function, which is included in the SQL:2003 standard and has been implemented by MySQL, Oracle Database, and Microsoft SQL Server.

To use `cast()`, you provide a value or expression, the `as` keyword, and the type to which you want the value converted. Here's an example that converts a string to an integer:

```
mysql> SELECT CAST('1456328' AS SIGNED INTEGER);
+----------------------------------+
| CAST('1456328' AS SIGNED INTEGER) |
+----------------------------------+
|                          1456328 |
```

```
+----------------------------------+
1 row in set (0.01 sec)
```

When converting a string to a number, the `cast()` function will attempt to convert the entire string from left to right; if any nonnumeric characters are found in the string, the conversion halts without an error. Consider the following example:

```
mysql> SELECT CAST('999ABC111' AS UNSIGNED INTEGER);
+--------------------------------------+
| CAST('999ABC111' AS UNSIGNED INTEGER) |
+--------------------------------------+
|                                  999 |
+--------------------------------------+
1 row in set, 1 warning (0.08 sec)

mysql> show warnings;
+---------+------+-----------------------------------------------+
| Level   | Code | Message                                       |
+---------+------+-----------------------------------------------+
| Warning | 1292 | Truncated incorrect INTEGER value: '999ABC111' |
+---------+------+-----------------------------------------------+
1 row in set (0.07 sec)
```

In this case, the first three digits of the string are converted, whereas the rest of the string is discarded, resulting in a value of `999`. The server did, however, issue a warning to let you know that not all the string was converted.

If you are converting a string to a `date`, `time`, or `datetime` value, then you will need to stick with the default formats for each type, since you can't provide the `cast()` function with a format string. If your date string is not in the default format (i.e., `YYYY-MM-DD HH:MI:SS` for `datetime` types), then you will need to resort to using another function, such as MySQL's `str_to_date()` function described earlier in the chapter.

Test Your Knowledge

These exercises are designed to test your understanding of some of the built-in functions shown in this chapter. See Appendix B for the answers.

Exercise 7-1

Write a query that returns the 17th through 25th characters of the string `'Please find the substring in this string'`.

Exercise 7-2

Write a query that returns the absolute value and sign (`-1`, `0`, or `1`) of the number `-25.76823`. Also return the number rounded to the nearest hundredth.

Exercise 7-3

Write a query to return just the month portion of the current date.

Previous chapter
< 6. Working with Sets

Next chapter
8. Grouping and Aggregates >